UNIVERSITY OF TRENTO

Master's Degree in Data Science

~ · ~

2025-2026

Secure LLM orchestration for enterprise workflows using the Model Context Protocol

Supervisor
Prof. Daniele Miorandi

Graduate Student
Ettore Miglioranza
257311

Final examination date: August 6, 2026

Dedica

Acknowledgments[1]

[1] In the interest of transparency, the author discloses that portions of the written text of this thesis were drafted and edited with the assistance of a generative artificial intelligence system, used as a writing aid under the author's continuous direction, review, and final approval. The research problem, system architecture, design decisions, implementation, and experimental validation presented in this work are the author's own. The author takes full responsibility for the accuracy and content of the entire document.

Abstract

Large language models (LLMs) have demonstrated cutting-edge capabilities in understanding intent and semantic routing, but their adoption in business workflows remains limited by stringent privacy and security requirements. This thesis investigates how LLM-based orchestration can be implemented in a real-world business context, through an architecture designed to keep personally identifiable information (PII) from crossing the boundaries of an external model in cleartext, a design objective rather than a formal guarantee, since entity detection is probabilistic. The work focuses on a specific workforce management domain within the Antares ERP ecosystem, where natural language requests must be reliably mapped to operational actions, such as locating nearby workers on client sites, identifying colleagues based on proximity, and validating absence cover and substitution chains.

The central research question is: how can an external LLM be integrated as a semantic router without exposing sensitive business data, whilst ensuring deterministic and verifiable execution of business tools? To answer this question, the thesis proposes a 'privacy-by-design' architecture based on the Model Context Protocol (MCP). The system enforces a strict separation between language understanding and data access through a reversible anonymisation pipeline: user requests are pseudonymised before reaching the model, routed by the LLM using only anonymised text, and de-anonymised exclusively on the server side at the time of tool execution. Entity detection is performed by a multilingual NER microservice based on GLiNER, chosen over rule-based alternatives such as Microsoft Presidio precisely because of its robustness in processing business inputs in Italian. Token-to-entity mappings are stored in a Redis context and are fully reversible via server-side deanonymisation; no time-to-live is currently configured on that context, a prototype-scope limitation this thesis states explicitly rather than leaving implied. The MCP server exposes schema-validated, typed tools to the model and enforces strict routing constraints, whilst a `.NET 10` client orchestrator manages the end-to-end request lifecycle.

The approach draws on the broader state of the art in privacy-preserving LLM infrastructure. OpenAI's Privacy Filter (Apache 2.0, 2025), recently made open source, confirms that lightweight, on-premise NER models incorporated as pre-processing layers represent an emerging pattern for PII containment; it performs one-way redaction, however, discarding the mapping between placeholders and entities once masking is complete. Reversibility on its own is not unique to this work, Presidio supports it too, through a dedicated encrypt/decrypt operator pair. What this thesis contributes relative to that broader landscape is the combination: local detection, reversible typed tokens, external semantic routing, and deterministic server-side execution operating together end to end, preserving full operational correctness whilst allowing controlled de-anonymisation at the protocol boundary, a distinction relevant under Article 4.5 of the GDPR, which classifies such processing as pseudonymisation rather than anonymisation.

The thesis makes three concrete contributions: (1) a multi-service architecture that structurally isolates sensitive data from the LLM context, (2) a deterministic and reversible privacy layer based on token mapping that preserves the correctness of business logic throughout the entire request lifecycle, and (3) a set of realistic workforce management use cases exercised end to end against a mock dataset standing in for the production Miorelli backend. The system described throughout this thesis is a validated prototype rather than a production deployment. Chapter 5 traces each use case in detail and states plainly what was, and was not, measured about semantic routing accuracy, NER performance, and residual PII in the payload reaching the LLM.

This work positions MCP as a practical and applicable interface standard for enterprise

AI orchestration and highlights the key engineering trade-off inherent in privacy by design: increased architectural complexity is the necessary cost for demonstrable privacy assurance, but it unlocks reliable and verifiable LLM-driven workflows without violating data protection constraints.

Keywords: LLM orchestration, Model Context Protocol (MCP), privacy-by-design, PII pseudonymization, semantic routing, named entity recognition, GLiNER, enterprise AI security, GDPR compliance

Contents

Chapter 1

Introduction

1.1 Context and Motivation

Large language models have demonstrated strong capabilities in interpreting user intent and routing natural-language requests to the right downstream action, capabilities that make them an attractive interface layer for enterprise software. Adoption in real business workflows has lagged behind that promise, and the reason is not technical performance. It is that most enterprise data, personnel records, financial figures, client information, is protected by data protection law and internal policy in ways that make sending it to an external, third-party model provider unacceptable by default. The gap between what LLMs can do and what enterprises are willing to let them see is where this thesis sits.

That gap is concrete, not abstract, in workforce management platforms. Antares/Miorelli is an ERP system used across the Italian facility management sector to manage personnel assignments to client sites, absence and leave requests, and the reporting relationships between workers and their managers. Users interact with it through structured web forms: selecting a site from a dropdown, filtering a worker list, filling in a ticket type. An LLM could replace that friction with a single natural-language question, interpreted directly and mapped onto the same underlying operations, without a rule engine or a trained classifier standing in between. But every one of those operations touches personnel data, and routing the request through a commercial LLM API means, on the face of it, sending that data outside the company's boundary.

This thesis was carried out as a professional internship at Delta Informatica / Saidea / Diventa S.r.l., an ICT services company based in Trento, within its Research and Development team, and it uses the company's Antares AI project as the concrete setting in which to work out an answer. The rest of this chapter states the problem precisely and the objectives that follow from it.

1.2 The Problem of Data Privacy in LLM Integration

The data behind every Antares/Miorelli request is personnel-sensitive by construction: worker names, phone numbers, residential coordinates, absence records, and the reporting relationships between employees and managers. All of it is personal data under the GDPR. Sending a raw user prompt such as *"Trova i colleghi di Giulia Conti"* to an external LLM provider means transmitting a real name, and by extension a real person's employment relationships,

outside the company's infrastructure and into a third-party service the company does not control. Management set an explicit constraint at the start of the project: no such data may leave the company's boundary in a form an external party could read.

That constraint rules out the simplest integration path. It does not rule out LLM-based routing itself, but it forces a specific architectural question: how a model can classify the intent of a request and extract its parameters without ever seeing the values those parameters refer to. The chapters that follow show that the answer is not to avoid PII exposure through policy or through trust in a vendor's data handling terms, but to reduce that exposure by design: replace every identifying value with a reversible token before the request reaches the model, and restore the original values only inside the company's own trusted boundary, at the point where the business operation actually executes.

Chapter 2 shows that existing tools, such as rule-based detectors and one-way redaction models, do not yet close this gap.

1.3 Research Objectives and Questions

The central question this thesis investigates is how an external LLM can be integrated as a semantic router without exposing sensitive business data, while still guaranteeing deterministic, verifiable execution of the underlying business logic. Answering it inside a real deployment setting means meeting three conditions at once: the LLM must handle the semantic interpretation of natural-language workforce requests, the architecture must minimise, by design, the risk that personally identifiable information crosses into the model's context, and the results produced must still be operationally correct, results a human operator could act on without a second, manual verification pass. Those three conditions break down into the more specific questions addressed in the chapters that follow.

The first question is architectural: what division of responsibility between client, anonymisation layer, LLM, and business-logic server structurally minimises PII exposure rather than merely forbidding it by policy. The second concerns the detection layer itself: whether a zero-shot, multilingual named-entity recognition model can reach the recall required for real Italian-language business text, where the rule-based alternative had already been tried and had failed. The third is about consequences: whether an architecture built around reversible anonymisation can still deliver accurate results on real use cases, or whether the privacy layer costs something in correctness.

1.4 Contributions

This thesis makes three concrete contributions, positioned against the state of the art reviewed in Chapter 2. Individually, rule-based PII detection, reversible token mapping, and LLM-based semantic routing are each established techniques on their own; Section 2.4 finds no reviewed system that combines local detection, reversible typed tokens, external semantic routing, and deterministic server-side execution end to end.

A privacy-by-design MCP architecture. It presents a four-service system, an anonymisation API, a GLiNER-based NER microservice, an MCP server, and an MCP client, structured so that the central design objective, keeping PII out of the LLM's context in cleartext, is pursued through the topology of the system rather than left to a policy the model is merely asked to follow.

A reversible tokenisation pipeline. It describes the design and the reasoning behind a token-based anonymisation scheme that preserves full operational correctness while allow-

ing controlled de-anonymisation at the protocol boundary, including the decision to replace an initial Presidio-based implementation with a GLiNER-based one after Presidio's recall on Italian business text proved insufficient for the company's real use cases.

Empirical validation on real workforce operations. It reports the implementation and end-to-end validation of three business use cases, nearest-worker lookup, colleague identification, and holiday request analysis, each exercised against a realistic mock dataset that reproduces the complexity of the production platform.

Validation throughout this thesis is carried out against a realistic mock data provider rather than the production Miorelli backend, a scope decision made jointly with the host company to prioritise the correctness of the privacy layer within the internship timeframe.

1.5 Structure of the Thesis

The rest of the thesis follows the conventional structure of an engineering thesis [11]: prior work motivates and constrains the design, the design is described in full before it is validated, and the results are discussed critically before drawing conclusions. Chapter 2 reviews the literature this work builds on: LLM integration into enterprise workflows, the Model Context Protocol, and privacy-preserving NLP, closing with the state-of-the-art gap that motivates the architecture. Chapter 3 presents that architecture in detail, from the privacy-by-design principles that govern it to the four-service runtime topology and the lifecycle of a single request. Chapter 4 describes the concrete implementation: the technology stack, the anonymisation pipeline's internals, the MCP tool layer, and the mock-backed Miorelli integration. Chapter 5 documents the three use cases and the validation carried out against them. Chapter 6 discusses the results, the design trade-offs made along the way, and the work left for a future iteration. Chapter 7 closes the thesis with a summary of what was achieved against the objectives stated in Section 1.3.

Chapter 2

Background and Related Work

This chapter reviews the three bodies of literature that inform the design of the system presented in subsequent chapters: the integration of large language models into enterprise software, the Model Context Protocol as a standardisation layer for LLM tool interfaces, and privacy-preserving natural language processing with a focus on named entity recognition and PII protection. Section 2.1 examines how large language models change semantic routing and tool invocation in enterprise systems, and the engineering constraints that shift introduces. Section 2.2 turns to the Model Context Protocol and the security boundaries it defines between client, server, and model. Section 2.3 addresses the regulatory and technical grounding of PII protection, from the GDPR's definition of personal data to detection approaches ranging from rule-based systems to zero-shot transformers. Section 2.4 surveys the current state of the art in secure LLM orchestration and identifies the gap that motivates this work.

2.1 Large Language Models in Enterprise Workflows

The modern generation of large language models traces its architectural lineage to the transformer, introduced by Vaswani et al. [30] in 2017. The attention mechanism, which allows each token in a sequence to attend to every other token, proved to be the basis for a qualitative leap in language understanding. Scaling transformer-based models, exemplified by GPT-3 [6] with its 175 billion parameters, showed that sufficiently large models exhibit capabilities not present in smaller counterparts: multi-step reasoning, few-shot generalisation from in-context examples, and reliable instruction following across diverse domains [31].

For enterprise software, the practical implication is a new paradigm for human-system interaction. Until recently, bridging the gap between a user's natural language expression and a system's formal data operations required rule engines, supervised intent classifiers, or custom NLP pipelines. A general-purpose language model can close that gap from the prompt alone, interpreting natural language requests and mapping them to well-typed business operations without labelled training data or domain-specific engineering.

The shift creates real engineering problems. Enterprise workflows demand deterministic, auditable outputs. Sensitive business data must not be transmitted to third-party model providers. System behaviour must remain predictable under adversarial or malformed inputs. These constraints sit in direct tension with the generative, probabilistic nature of LLMs, and reconciling them is an active area of research and engineering.

2.1.1 Semantic Routing and Intent Classification

Semantic routing refers to interpreting a natural language utterance and directing it to the appropriate downstream component based on its inferred intent. Classical approaches used supervised intent classifiers trained on labelled utterance datasets [17]. These models required curated training data per domain and were brittle under paraphrasing, multilingual input, or out-of-distribution requests. In practice, that limitation has a direct cost: adding a new business action, or supporting a new language, means collecting and labelling a fresh batch of example utterances before the classifier can be retrained and redeployed, a cost that scales with every intent and every language a deployment needs to cover.

LLMs remove that cost. Because they acquire broad linguistic knowledge during pre-training, they can classify intent zero-shot given only a natural language description of the available actions, with no labelled dataset and no retraining step. Routing logic becomes a prompt rather than a trained artefact, and adding a new action or language is a matter of editing text, not collecting data. A recent example, though outside classic intent classification, is a conversational, RAG-based assistant for AI-supported document creation workflows [9], which replaces a rule-based document generation flow with natural-language interaction end-to-end. The saving is not free, however. The quality of zero-shot routing depends heavily on the specificity and clarity of tool descriptions, and vague or overlapping descriptions cause the model to route incorrectly regardless of the underlying model's capability, making prompt engineering a first-class concern in deployed systems.

2.1.2 Tool Calling and AI Agents

The capacity for LLMs to invoke external functions (also called function calling) was formalised as a first-class API feature by major model providers in 2023. In this paradigm, the developer supplies the model with a schema describing callable functions, their parameters and return types. The model produces a structured call rather than free text; the host application executes it and feeds the result back for the model to incorporate into its response.

The ReAct framework [32] formalised the interleaving of reasoning traces and tool invocations, showing that a model that explicitly reasons about which tool to call outperforms models that invoke tools without intermediate reasoning, especially in multi-step tasks. This is the AI agent paradigm: a language model acting as an orchestrator that selects and sequences tool calls to reach a higher-level goal.

The Toolformer work [27] demonstrated that models can learn to use external APIs from self-supervised examples, acquiring tool use directly from patterns in text rather than from an explicit interface definition. Nothing in that training signal constrains the resulting call to a fixed, machine-checkable format: a malformed or incomplete invocation is not distinguishable, from the model's own output, from a well-formed one. That gap is not cosmetic in an enterprise setting. A routing decision that cannot be validated against a schema cannot be logged in a structured, replayable form, and there is no reliable way to confirm after the fact which tool was called with which parameters. Later systems such as Gorilla [24] narrowed the correctness gap by fine-tuning a model on API documentation and evaluating calls by execution rather than surface similarity, but the underlying call is still free text scored after the fact, not structurally guaranteed beforehand. For this reason, enterprise deployments generally prefer explicit schema-driven invocation over learned tool use. Schema-driven invocation provides stronger output format guarantees and makes the routing decision auditable.

Two architectural properties of tool-calling systems have direct implications for enterprise deployments. First, the model's agency can be bounded by restricting which tools it may call: exposing only business-level operations and keeping data access inside deterministic code limits the model to intent classification and parameter extraction. Second, the tool result feedback loop, in which tool outputs are returned to the model for further reasoning, is the primary vector for indirect prompt injection attacks [14], making its design a security-relevant decision.

2.2 The Model Context Protocol (MCP)

The Model Context Protocol [2], published by Anthropic in November 2024, standardises the communication interface between LLM client applications and external tool servers. Before MCP, each LLM-tool integration required bespoke glue code, custom serialisation formats, and proprietary conventions. MCP defines a vendor-neutral interface so that any compliant client can connect to any compliant server without integration-specific adaptation.

The design is modelled after the Language Server Protocol (LSP) [7], which eliminated the $N \times M$ explosion of editor-language integrations by decoupling language-analysis servers from editor clients. MCP applies the same logic: any client that speaks MCP can talk to any server that speaks MCP, reducing the integration surface from $N \times M$ to $N + M$.

2.2.1 Standardizing LLM Interfaces

MCP defines three categories of server-exposed primitives [2]. *Tools* are callable functions with typed input schemas and structured return values, analogous to OpenAPI operations. *Resources* are read-only data sources the model can pull into its context: file contents, database records, configuration objects. *Prompts* are reusable, parameterised message templates.

Communication runs over two transports: standard input/output for locally spawned server processes, and HTTP with Server-Sent Events (SSE) for network-accessible servers. The underlying message format is JSON-RPC 2.0. Tool schemas are JSON Schema objects describing parameter types, constraints, and documentation. This standardised schema representation allows tooling to auto-generate type-safe client code from server definitions, reducing the risk of schema drift between client expectations and server implementation.

2.2.2 Security Boundaries in MCP

MCP's security model separates three roles. The *MCP host* manages client connections and controls which servers the model may reach. The *MCP server* is a trusted executor with access to real data and infrastructure. The *model* is a semantic reasoner: it produces structured tool calls but accesses no data source directly.

This three-layer separation creates a natural enforcement point for privacy. The server is the only component that handles raw sensitive data. A pre-processing step applied to user requests before they reach the model can structurally prevent PII from entering the LLM context, because the server's trust boundary sits downstream of the model. The model operates only on what the client chooses to send.

Prompt injection [14] is the primary known attack on this architecture. Malicious content embedded in tool results or externally retrieved resources can attempt to hijack the model's behaviour by simulating system-level instructions. Mitigations include restricting the model's tool set, limiting the scope of requests the model may issue, and routing business

logic through deterministic code that does not re-enter the model after execution. These mitigations reduce the attack surface but do not eliminate it, and the design of the tool result feedback loop remains an open security concern in MCP deployments. A systematic security analysis of the MCP ecosystem [16] catalogues these risks across the full server lifecycle, from malicious tool registration to compromised transport, and finds that few real deployments implement the safeguards it recommends.

2.3 Privacy-Preserving Natural Language Processing

User requests directed at business systems routinely contain personally identifiable information: employee names, site addresses, phone numbers, absence dates. Under data protection regulations, transmitting this information to external model providers requires a valid legal basis and appropriate safeguards. This section reviews the regulatory context and the technical approaches developed to address PII exposure in NLP pipelines.

2.3.1 Personally Identifiable Information (PII) Definitions

The General Data Protection Regulation [12], Article 4(1), defines personal data as "any information relating to an identified or identifiable natural person." Identifiability is broad: a person is identifiable if they can be singled out by name, identification number, location data, or factors specific to their physical, professional, or social identity. Combinations of individually innocuous data points can identify an individual in aggregate.

Article 4(5) introduces *pseudonymisation*: processing personal data so that it can no longer be attributed to a specific person without additional information, provided that additional information is kept separately with appropriate technical safeguards. Pseudonymisation is distinct from anonymisation. Pseudonymised data remains personal data under the GDPR, because the mapping between pseudonyms and identities is preserved, even if stored separately. Truly anonymous data cannot be re-linked by any reasonably likely means and falls outside the regulation entirely.

This distinction has direct engineering implications for privacy-preserving LLM pipelines. A system that replaces PII with reversible tokens before sending data to an external model provider is performing pseudonymisation, not anonymisation. The privacy guarantee it provides is a limitation of PII exposure to the model, not its elimination. The token-to-entity mapping must itself be protected and handled as personal data under the GDPR.

2.3.2 Named Entity Recognition (NER) for Data Masking

Named entity recognition is the NLP task of identifying and classifying contiguous spans of text that refer to named entities: persons, organisations, locations, dates, phone numbers. For PII protection, NER is the detection layer of a two-stage pipeline: entities are identified, then replaced with neutral placeholders that preserve syntactic structure without revealing identifying content.

Classical NER systems based on Conditional Random Fields or bidirectional LSTMs required annotated corpora per language and domain. Pre-trained transformer encoders, BERT [10] and its multilingual variants, reduced data requirements substantially by enabling fine-tuning on small labelled datasets. Domain and language specificity remained a persistent challenge. A model trained on English newswire won't reliably handle Italian business communications.

The error asymmetry in masking pipelines is architecturally significant. A false negative (a missed PII span) directly violates the privacy guarantee: the unmasked entity crosses the model boundary in plaintext. A false positive (a masked non-PII span) degrades the utility of the downstream task but creates no privacy breach. This asymmetry implies that detection components should be optimised for recall, with post-processing to suppress common false positives.

The design of replacement tokens also affects downstream task quality. Replacing every entity with a single generic token (e.g., `[REDACTED]`) destroys structural information the model may need: if a request mentions two different people and both become the same token, the model cannot distinguish them when making a routing decision. Label-typed, numbered tokens such as `<PERSON_0001>` and `<PERSON_0002>` preserve entity type and cardinality while removing identifying content, enabling correct multi-entity routing even when all PII has been replaced.

2.3.3 Rule-based vs. Deep Learning Approaches (Presidio vs. GLiNER)

Two broad approaches are available for PII detection in NLP pipelines: rule-based systems that combine regular expressions, gazetteers, and lightweight statistical models; and transformer-based systems that apply encoder models to span classification over arbitrary text.

Microsoft Presidio [21] is the leading open-source rule-based framework. It provides recognisers for names, phone numbers, email addresses, credit card numbers, and national identifiers, built on regular expressions and context-sensitive matching, optionally extended with spaCy NER models. Its strengths are predictability, low computational cost, and an extensible plugin architecture. Its weaknesses are language dependence (most recognisers target English) and rigidity: entity types that don't match predefined patterns are silently missed, with no indication of failure. Extending coverage to a new language or entity type requires writing explicit rules, which is labour-intensive and inherently incomplete.

GLiNER [34] (Generalist Model for Named Entity Recognition) is a zero-shot NER model built on a bidirectional transformer encoder. Instead of training a fixed classification head per entity type, GLiNER conditions detection on natural language label descriptions given at inference time. The model computes a similarity score between the label description and each candidate span, enabling detection of arbitrary entity types without fine-tuning. The multilingual variant `urchade/gliner_multi-v2.1`, trained on data spanning multiple European languages, provides coverage of morphologically rich languages such as Italian, including proper name patterns and domain-specific vocabulary that rule-based approaches miss.

Compared to Presidio, GLiNER offers three concrete advantages for multilingual and domain-diverse deployments. The detection label set is configurable at runtime without model retraining. Italian-language support is native, not an adaptation. Each prediction carries a confidence score, enabling a configurable threshold to tune the precision-recall trade-off. A post-processing pipeline over raw GLiNER predictions (span normalisation, stopword filtering, overlap deduplication) can further improve precision without sacrificing recall on genuine PII.

2.4 State of the Art in Secure LLM Orchestration

The dominant pattern in privacy-sensitive LLM deployments is a *privacy-preserving proxy*: a pre-processing layer that sanitises user input before it reaches the model. Early implemen-

tations used regular expression substitution, adequate for structured PII (phone numbers, email addresses) but inadequate for unstructured natural language. More recent work applies transformer-based NER, trading computational cost for substantially better recall on diverse inputs.

Three systems anchor the current state of the art for PII detection in LLM pipelines: Microsoft Presidio, the rule-based baseline discussed in Section 2.3.3; GLiNER, the zero-shot transformer detector adopted in the architecture presented in this thesis; and OpenAI's Privacy Filter, reviewed in detail below as the most complete open-weight alternative. A related concern shows up in adjacent academic work: Mansillo [18] anonymises synthetic training data inside a multimodal retrieval-augmented pipeline, which confirms that anonymisation-aware LLM systems are an active area of interest beyond this thesis, though that work does not address reversibility or tool-execution correctness either. Two recent papers push in the same direction from different angles. Hou et al. [15] propose a general pseudonymisation framework for cloud-based LLMs that replaces private information before a remote API call, evaluated for privacy and utility but not for downstream tool-execution correctness. Albanese et al. [1] anonymise text on-premise with a locally run LLM before it reaches a question-answering agent, prioritising factual utility over the reversibility this thesis requires. The three systems diverge along the dimensions that matter for an enterprise LLM orchestration system, and no single one of them covers all of them.

All three run entirely on local infrastructure and ship under a permissive licence, MIT for Presidio and Apache 2.0 for the other two, so neither dimension separates them.

Multilingual coverage is uneven, however. Presidio's recognisers are English-centric and need new rules for each additional language, GLiNER's multilingual variant handles Italian natively, and Privacy Filter's non-English performance is undocumented and, per its own model card, may degrade on morphologically rich languages. Reversibility separates them further. Presidio can be reversible in principle through a dedicated encrypt/decrypt operator pair, but that ties reversibility to key management external to the detection pipeline. GLiNER is a detector with no built-in notion of a token-to-entity mapping; reversibility, where it exists, is a property of the system built around it, not of GLiNER itself. Privacy Filter performs one-way redaction by design and discards the mapping entirely. On detection accuracy, Privacy Filter is the only one of the three to publish a comparable accuracy figure, discussed in detail in Section 2.4.1. Presidio and GLiNER have no comparably reported number. Presidio is tuned for precision over broad recall by construction, and GLiNER is tuned for recall across arbitrary label sets, so neither figure would be comparable to Privacy Filter's self-reported one even if it existed.

None of the three systems combines reversibility with a validated benchmark for operational correctness. The one system with a quantitative accuracy figure, Privacy Filter, is also the one that discards the token-to-entity mapping outright. The systems that could in principle preserve or approximate that mapping, Presidio and GLiNER, report no comparable benchmark of their own.

The literature addresses detection and masking largely in isolation. End-to-end systems that detect PII, mask it before the model boundary, execute business operations using real values server-side, and return a safe response to the client remain uncommon in published work. Most published systems stop at the masking step and do not address the operational requirement that tool execution needs the original entity values, not their placeholders.

2.4.1 OpenAI Privacy Filter: A Reference Open-Weight Model for PII Redaction

The most significant open-weight contribution to this space is OpenAI's Privacy Filter [23], released in April 2026 under the Apache 2.0 licence. It is the first publicly available, production-quality PII detection model explicitly designed as a pre-processing layer for LLM pipelines.

Architecture. Privacy Filter is a bidirectional token-classification model derived from the GPT OSS family. The autoregressive generation head is replaced by a classification head over a BIOES span-labelling scheme with 33 output classes. A constrained Viterbi decoder enforces coherent span boundaries across multi-token entities. The Sparse Mixture-of-Experts design gives 1.5 billion total parameters but only approximately 50 million active per token, enabling high-throughput inference at low cost.

Capabilities. The model detects eight PII categories: `private_person`, `private_address`, `private_email`, `private_phone`, `private_url`, `private_date`, `account_number`, and `secret`. Its context window is 128,000 tokens. On the PII-Masking-300k benchmark it achieves 96% F1 (97.43% on the corrected version). Fine-tuning on small in-domain datasets can push domain-specific performance from 54% to 96% F1 quickly, suggesting a strong general-purpose foundation.

Local execution. The model is designed to run on-premise, within the organisation's infrastructure, removing the need for a Data Processing Agreement with a cloud vendor and addressing GDPR data residency requirements directly.

One-way redaction. Privacy Filter performs one-way redaction: the placeholder tokens it produces carry no mapping back to the original entities. This is appropriate for use cases where the model's output does not need to reference original PII values, such as summarisation, classification, and general question-answering. For enterprise workflows in which a routing decision must trigger operations on real, identified data, one-way redaction is insufficient. A system that routes a request mentioning `<LOCATION_0001>` must know what `<LOCATION_0001>` is at the point of tool execution.

OpenAI's model card acknowledges that Privacy Filter performs pseudonymisation under GDPR Article 4(5), not anonymisation. The placeholder tokens combined with the original document could in principle be reversed by a party holding both. This is the correct framing of what such systems achieve: they limit PII exposure to the model provider, but the mapping between tokens and entities still exists and must be managed.

Multilingual limitations. Privacy Filter's performance on non-English languages is not officially documented and may degrade significantly on morphologically rich languages such as Italian.

The body of work reviewed in this chapter establishes that privacy-preserving pre-processing is the right pattern for LLM orchestration in sensitive enterprise domains. What the existing systems do not address is the combination of reversibility with operational correctness: enabling a routing system to work on pseudonymised inputs while guaranteeing that tool execution uses the original entity values, without exposing those values to the model provider. This gap is the starting point for the architecture described in Chapter 3.

Chapter 3

System Architecture and Design

This chapter presents the architecture of Antares AI, a privacy-oriented LLM orchestration system for enterprise workforce management. Chapter 1 and Chapter 2 set out the problem this architecture answers: an LLM has to route natural-language workforce requests without seeing the personal data those requests refer to. The design that follows is organised around one central objective, keeping personally identifiable information out of the LLM context boundary in cleartext, and most of the decisions described in this chapter follow from what it takes to pursue that objective in practice, given that entity detection is probabilistic rather than certain. Section 3.1 sets out the privacy-by-design principles that govern every subsequent decision. Section 3.2 introduces the Antares/Miorelli domain and the constraints it places on the system. Section 3.3 translates those constraints into a traceable list of functional and non-functional requirements. Section 3.4 presents the four-service runtime architecture. Section 3.5 details the design of each component. Section 3.6 traces a complete request through the system end to end.

3.1 Architectural Principles: Privacy-by-Design

Privacy by design, formalised by Cavoukian [8] and later codified as a legal obligation in GDPR Article 25 [12], is the principle that privacy protection must be built into systems from the outset. Cavoukian's framework identifies seven foundational principles: proactive prevention of privacy harms, privacy as the default operational setting, privacy functionality embedded into the architecture, full functionality without zero-sum trade-offs, end-to-end security across the data lifecycle, visibility and transparency in system operation, and user-centric design. The Antares AI architecture applies these principles as structural design constraints, not as runtime policy.

Three of Cavoukian's principles shape the system most directly.

Privacy as the default. Anonymisation of user requests before LLM transmission is the only supported operational path. A client that skips the anonymisation step sends an unprocessed prompt, but the server system prompt still instructs the model to treat input as pseudonymised tokens; bypassing anonymisation is detectable and requires an explicit decision, not an oversight. Nothing in the system's configuration surface offers a way to switch privacy enforcement off.

Privacy embedded into design. PII isolation is not achieved through audit logging, access controls, or post-hoc output redaction. It is structural: the anonymisation service sits at the architectural boundary and replaces detected PII with tokens before any request

reaches the model, so the model is not deliberately given identifying data to work with. This protection holds only for entities the detection layer actually finds; an entity it fails to catch is never tokenised and passes through unchanged, a risk Chapter 5 addresses directly, whether by reporting measured detection performance or by stating explicitly that such measurement was not carried out.

End-to-end security. The token-to-entity mapping is stored in a session-scoped Redis context; on the path where a business operation completes, it is consumed exactly once during server-side deanonymisation and removed. That removal is not currently guaranteed independent of that path: no time-to-live is configured on the context, so a mapping tied to a request that errors out before deanonymisation persists in Redis indefinitely, a prototype-scope limitation discussed in Chapter 6. Operational reports written to disk by the `JobTools` layer, containing real names and phone numbers, stay on server-local storage and are never sent to the client, but are themselves a persistent, unencrypted record with no retention policy applied.

Together these constraints define the *architectural design objective*: PII should exist in the system only within the server's trusted execution boundary, and only for the duration of one business operation.

3.2 The Antares AI Ecosystem

Antares AI was developed within the Antares/Miorelli ecosystem, an ERP platform for workforce management used in the Italian facility management sector. The platform manages personnel assignments to job sites (*cantieri*), tracks absence and leave requests, and maintains the hierarchical relationships between workers and managers. Users currently interact via structured web interfaces; Antares AI enables the same operations through natural language, routing each request to the correct business tool automatically.

The domain is demanding on privacy and correctness for concrete reasons. The data is personnel-sensitive by nature: names, phone numbers, residential coordinates, absence records, and employment relationships are all PII under the GDPR. The user base is multilingual (requests arrive in Italian, regional dialects, or a mix), which ruled out recogniser-based NER tools designed for English. Most critically, the operational context requires accurate results: a holiday request analysis that produces an incomplete result is operationally harmful, not just inconvenient, because it may leave a client site under-staffed. The system must preserve full business logic accuracy even when anonymising inputs.

Workforce operations are structured around three core entity types: *persone* (workers), *cantieri* (client sites), and *ticket* (absence and leave records). Relations are typed: `PULITO_DA` links a worker to their assigned sites, `RESPONSABILE_DI_RIFERIMENTO` identifies a worker's manager, and `AUTORIZZATORE` identifies the person authorised to approve leave. These entity types and relation names appear in natural language requests and define the PII surface the anonymisation pipeline must cover.

3.3 Functional and Non-Functional Requirements

The principles and domain constraints introduced above translate into a concrete set of requirements the architecture must satisfy. This section makes them explicit and traceable, without repeating the discussion already given in Sections 3.1 and 3.2.

Functional requirements follow directly from the three use cases the system supports, each one a concrete operation a dispatcher or site manager already performs by hand. Given

a named client site, the architecture has to find the nearest available workers within a configurable radius and rank them by distance (Use Case A, traced in detail in Section 3.6). Given a worker's name, it has to identify that worker's direct colleagues and surface a ranked pool of nearby substitutes (Use Case B, Section 4.4.2). Given a holiday request, it has to resolve the full chain end to end: verify the requester's authorisation, determine which colleagues are absent on the relevant dates, and identify who is available to cover (Use Case C, Section 4.4.3).

Non-functional requirements follow from the domain constraints described in Section 3.2, and each one is really a boundary condition on the functional requirements above rather than an independent property. The privacy objective from Section 3.1 sets the design target: PII should not cross the LLM context boundary in cleartext, for any request pattern the anonymisation layer correctly handles. Operational accuracy sets a second one, in the opposite direction, since a routing or entity-resolution error introduced by the privacy layer is not an acceptable trade-off when an incomplete result can leave a client site under-staffed. Multilingual support is the third: the system has to handle Italian as a native language, not as an adaptation of an English-first pipeline, given how the actual user base writes.

3.4 High-Level System Architecture

Antares AI comprises four cooperating runtime services, each with a distinct responsibility and a well-defined interface. Figure 3.1 illustrates the runtime topology and data flows.

The four services and their roles are summarised in Table 3.1.

Table 3.1: Antares AI runtime services

Service	**Stack**	**Port**	**Role**
MCP Client	C#/.NET 10	—	Request orchestration and test harness
Anonymization API	C#/.NET 10	5292	PII detection, tokenisation, context storage
GLiNER Service	Python/FastAPI	5100	Named entity recognition
MCP Server	C#/.NET 10	5001	LLM routing and business tool execution

The communication topology mirrors the system's trust model. The MCP client is an untrusted endpoint: it never receives real personnel data, and the only thing it sees is a hardcoded Italian confirmation string. The anonymisation API sits at the client-side boundary, processing the raw prompt and issuing a session token (`contextId`) that indexes the stored mapping. The GLiNER service is a stateless NER oracle; it knows nothing about tokens, Redis, or business context. The MCP server is the trusted execution boundary: it receives only anonymised prompts, routes them via Claude, restores PII server-side, and executes business tools against real data.

Redis [26] runs at port 55000 as the context store. It is not reachable from the client and is accessible only to the anonymisation API (writing) and the MCP server (reading and consuming).

The architectural design objective (keeping PII out of the MCP boundary) is pursued at the interface between client and server. The `ask_antares` tool is designed to accept only an anonymised prompt and a `contextId`; whether it in fact receives raw PII depends on the anonymisation layer having detected it upstream. The client calls the anonymisation API first, gets an anonymised prompt and a `contextId`, and only then constructs the tool call. PII restoration from the `contextId` happens inside the server's trusted boundary, after the routing decision has been made.

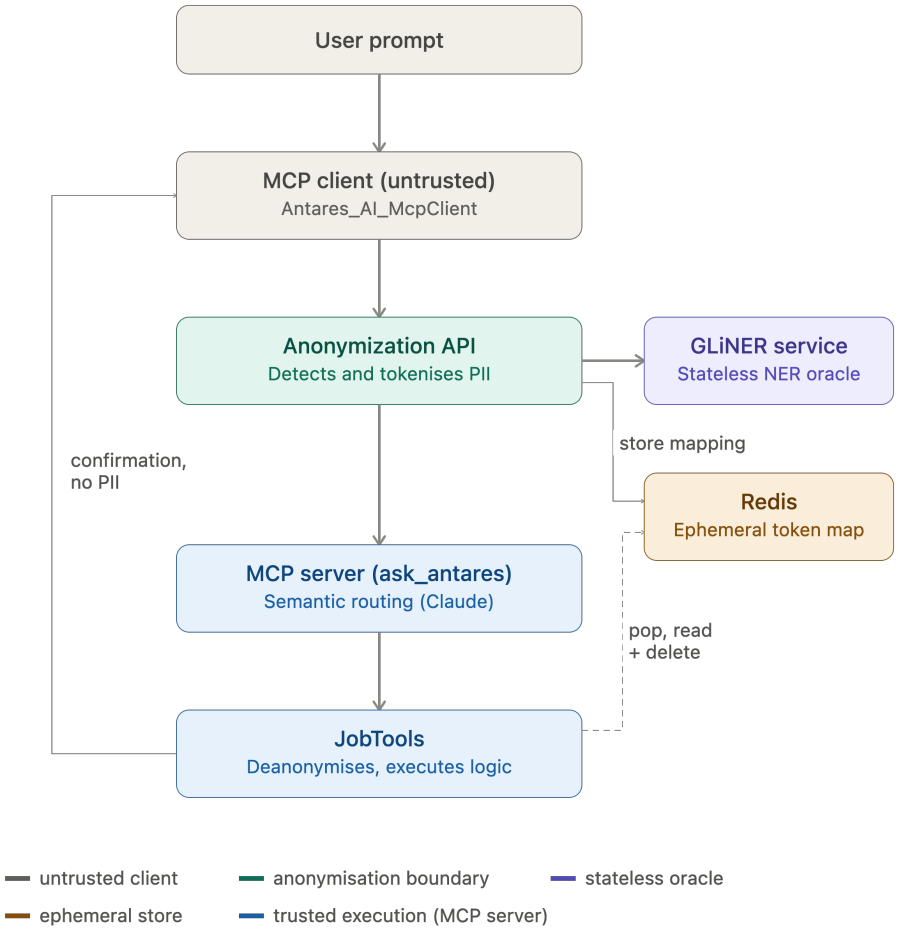

Figure 3.1: Runtime architecture of Antares AI. The client anonymises the user prompt before crossing the MCP boundary. The MCP server routes the anonymised request via Claude and executes the selected business tool server-side, where PII is restored from the Redis context. The client receives only a hardcoded confirmation string.

3.5 Component Design

3.5.1 The MCP Client Orchestrator

The MCP client (`Antares_AI_McpClient`) is a .NET 10 [20] console application that orchestrates the end-to-end request lifecycle. In the current system it serves as both the reference client implementation and the integration test harness for the three use case scenarios.

Its responsibilities are deliberately minimal. It constructs a natural language prompt, submits it to the Anonymization API, and then calls `ask_antares` on the MCP server with the anonymised prompt and `contextId`. The client performs no business logic and holds no sensitive data; it can't observe PII because it never calls a deanonymisation endpoint or touches the Redis context.

Communication with the MCP server uses the HTTP/SSE transport via `ModelContextProtocol.Core`. Communication with the Anonymization API uses a strongly typed HTTP client generated by NSwag [29] from the API's OpenAPI specification, keeping the client's view of the API interface in sync with the server implementation automatically.

A production client would accept dynamic user input, manage multi-turn context across `contextId` sessions, and surface token placeholders in the UI as opaque references. The current implementation uses hardcoded prompts to enable deterministic integration testing.

3.5.2 The Anonymization Service API

The Anonymization Service (`AnonymizationServiceAPI`) is an ASP.NET Core Minimal API application that acts as the PII gateway between client and server. It exposes three REST endpoints: `GET /anonymize`, `POST /anonymize-into-context`, and `POST /deanonymize-known`.

The primary client-facing operation is `/anonymize`. It receives a raw prompt, delegates entity detection to the GLiNER service, replaces detected entities with stable typed tokens, generates a `contextId`, stores the token-to-entity mapping in Redis, and returns the anonymised text, the `contextId`, and a detection count. The token format is `<LABEL_NNNN>`, where `LABEL` is the uppercase entity type and `NNNN` is a zero-padded counter per label per context. This preserves entity type and cardinality while stripping identifying content.

The `/anonymize-into-context` endpoint extends an existing session by merging new token mappings into an already-issued `contextId`. This supports multi-turn interactions where a follow-up request introduces a new entity that must be pseudonymised within the same operational context.

The `/deanonymize-known` endpoint is the server-facing operation. It receives an anonymised string and a `contextId`, retrieves the mapping from Redis via a `Pop` operation (atomic read-and-delete), and restores original values through string substitution. The one-time-use design limits the exposure window for the mapping to the minimum needed for tool execution; a repeated call with the same `contextId` will find no context and fail cleanly.

Detection is fully delegated to the GLiNER service. The separation between detection (GLiNER) and masking plus context management (Anonymization API) preserves substitutability: the NER backend can be replaced without touching the tokenisation logic.

3.5.3 The GLiNER-based NER Microservice

The GLiNER service (`GLiNERService`) is a Python FastAPI [25] application that exposes a single stateless endpoint, `POST /analyze`, and a health check at `GET /health`. Its job is to apply the GLiNER model to an input text and return a list of detected entity spans with labels, character offsets, and confidence scores.

The multilingual model `urchade/gliner_multi-v2.1` [34] is loaded once at startup and held in memory for the process lifetime. Model weights download from HuggingFace on first run and are cached locally; subsequent starts use the cached copy.

The service is a dumb NER oracle. It receives text and labels, runs the model, returns spans. All policy decisions (which labels to request, what confidence threshold to apply, how to handle overlapping spans) are made by the Anonymization API, which configures the `/analyze` call accordingly. This keeps the GLiNER service simple and reusable.

Three post-processing stages run within the Anonymization API before token substitution. Raw predictions are normalised: invalid offsets, missing labels, and out-of-bounds spans are discarded. Low-quality entities are filtered out: spans below the 0.45 confidence threshold, spans shorter than three characters, and spans matching an Italian stopword blocklist are removed. Overlapping predictions are deduplicated, keeping the higher-confidence, longer span. The result is a clean, non-overlapping list of entity spans ready for replacement.

3.5.4 The Antares MCP Server

The MCP server (`Antares_AI_McpServer`) is the trusted execution boundary. It exposes the `ask_antares` routing tool, calls the Claude API for semantic routing, and executes the selected business tool against the Miorelli data layer.

Tools are registered via `ModelContextProtocol.AspNetCore`, which generates MCP-compliant schemas from C# class and method annotations. Two types are registered: `AntaresTools` (exposing `ask_antares`) and `JobTools` (exposing the three business operations). The `JobTools` methods are not called directly by the MCP client; Claude invokes them through function-calling within an `ask_antares` execution.

The server calls Claude Sonnet 4.6 via the `Anthropic.SDK` client. The system prompt defines Claude's role as a semantic router, lists the available sub-tools with descriptions, and enforces privacy constraints: the model must not infer or reveal PII from tokens, must forward the `contextId` to the selected sub-tool, and must include the sub-tool's result string verbatim in its response. A retry loop with exponential backoff handles transient `overloaded_error` responses, retrying up to four times before returning an error message.

The data access layer is abstracted behind `IMiorelliService`. The current implementation registers `MiorelliServiceStub`, which throws `MiorelliClientNotAvailableException` on every call; each business tool catches this and falls back to static mock data from `MiorelliMockDataProvider`. Switching to the real Miorelli tenant changes one line in `program.cs`, registering `AntaresMiorelliClient` instead of the stub, with no tool code changes required; that line is necessary for the migration, not sufficient for it, since the client implementation itself, identity propagation, authentication, access control, and integration testing against the real tenant are still open work. Section 4.5.3 discusses this design and its remaining gaps in full.

3.6 Data Flow and Lifecycle of a Request

This section traces a user request from natural language prompt to confirmation response. Use Case A (finding the nearest available workers to a named client site) serves as the concrete example.

3.6.1 Tokenisation and Redis Context Storage

The lifecycle begins when the MCP client constructs a natural language prompt. For Use Case A, a typical prompt is: *"Trova i lavoratori più vicini al cantiere Bergamo Via della Fonda 5 Giulia S.r.l. entro 15 km. Risolvi tutti i GUID internamente."* The site descriptor *Bergamo Via della Fonda 5 Giulia S.r.l.* identifies both the client site and the contracting company operating it, and is the PII span the anonymisation layer must remove before this prompt crosses the MCP boundary.

The client calls `GET /anonymize`. The API calls the GLiNER service, receives entity predictions, runs the three-stage filtering pipeline, and sorts surviving entities by character offset in reverse order. Replacements are applied right-to-left to avoid invalidating offsets of earlier spans. GLiNER's zero-shot detection does not resolve this compound descriptor as a single location plus a single organization, as a rule-based recogniser tuned on it might; it surfaces three separate `location`-typed spans instead: *Bergamo*, *Via della Fonda*, and *Giulia S.r.l.*, with the street number `5` falling between spans and left untagged (not itself identifying). The `organization` label is never applied here, even though the anonymisation service's label set includes it. This is measured, reproducible behavior, not a one-off; Section 5.5.2 reports the full quantitative detection results this single trace is drawn from. The anonymisation service tokenises per predicted span rather than merging adjacent same-type detections, so three tokens are produced: `<LOCATION_0001>`, `<LOCATION_0002>`, and `<LOCATION_0003>`.

The mapping `{"<LOCATION_0001>": "Bergamo", "<LOCATION_0002>": "Via della Fonda", "<LOCATION_0003>": "Giulia S.r.l."}` is stored in Redis under `anon:context:{contextId}`, where `contextId` is a freshly generated UUID. The API returns the anonymised prompt, `"Trova i lavoratori più vicini al cantiere <LOCATION_0001> <LOCATION_0002> 5 <LOCATION_0003> entro 15 km..."`, and the `contextId` to the client. No PII is in the response; the client holds only an opaque session identifier. The fragmentation does not itself create a leak, since every span GLiNER did detect is tokenised regardless of how it is split or labelled — but it illustrates why detection recall, not label precision, is the property the privacy claim actually rests on.

Redis is the right tool here for practical reasons: its in-memory architecture gives sub-millisecond read and write latency, which matters in a synchronous request pipeline. Its native hash type matches the token-map structure directly, and its in-memory-with-optional-persistence model suits ephemeral session data that does not need to survive a service restart.

3.6.2 Isolated Semantic Routing

The client constructs the `ask_antares` tool call with the anonymised prompt and `contextId`, dispatching it over HTTP/SSE. At this point the MCP boundary has been crossed and no PII is in transit.

The `ask_antares` implementation builds a Claude API call. The system prompt defines Claude's role as an Italian-language semantic router, provides the JSON schemas of the available sub-tools, and specifies the expected response format. The anonymised user prompt becomes the user message.

Claude's task is intent classification (which sub-tool?) and parameter extraction (what values, in anonymised form?). For Use Case A it classifies the intent as `get_nearest_workers_to_cantiere` and extracts `cantiereName = "<LOCATION_0001>"` and `context_id = {contextId}`. It doesn't try to resolve the token; it works entirely with the pseudonymised representation.

The Claude API returns a structured tool call rather than free text. The `Microsoft.Extensions.AI` abstraction intercepts it, locates the registered `GetNearestWorkersToCantiere` method,

and invokes it with the extracted parameters. The Redis context is not touched during this phase.

3.6.3 Server-Side Deanonymisation and Tool Execution

The `GetNearestWorkersToCantiere` method is the first point where PII re-enters the system. Its first action is to call `DeanonymizeAsync`, which posts to `/deanonymize-known` with the token `"<LOCATION_0001>"` and the `contextId`. The API atomically reads and deletes the context from Redis and returns the original value: `"Bergamo Via della Fonda 5"`. The Redis context no longer exists after this call.

The deanonymised value drives the business logic entirely within the server's trusted boundary. The method resolves the caller's authorised cantieri via the Miorelli service layer, locates the matching site, retrieves assigned workers via the `PULITO_DA` relation, computes Haversine distances from each worker's registered coordinates to the site's GPS coordinates, and filters and ranks results by distance within the specified radius.

The full report, containing real names, phone numbers, distances, and status information, is written to a timestamped file in the server's local `logs/` directory. It is never transmitted to the client, though it remains on disk afterward as an unencrypted, retention-unmanaged record, the same limitation named in Section 3.1. The method returns a hardcoded Italian confirmation string: *"Operazione completata con successo."* Claude includes this verbatim in its final response.

The client receives a short text block with no PII. For this trace, no personally identifiable information crossed the MCP boundary, and a complete, accurate operational result was produced server-side; Chapter 5 addresses how consistently that holds beyond this single worked example, whether through measured results or as an acknowledged limitation of the validation carried out.

Chapter 4

Implementation Details

Chapter 3 defined the architectural design objective that governs Antares AI: keeping personally identifiable information out of the LLM context boundary in cleartext. This chapter shows how that objective is pursued in running code. Section 4.1 describes the technology stack and the reasoning behind splitting the system across two language ecosystems. Section 4.2 covers the two implementation phases of the reversible anonymisation pipeline, from an early Presidio-based prototype to the GLiNER-based system now in use. Section 4.3 details how business operations are exposed to Claude as MCP tools. Section 4.4 describes the orchestration logic behind the three use cases. Section 4.5 closes the chapter with the abstraction layer that separates business logic from the Miorelli backend it currently mocks.

4.1 Technology Stack and Development Environment

Antares AI is split across two language ecosystems. The MCP server, the MCP client, and the Anonymisation Service are all built on .NET 10 with ASP.NET Core [20]. The GLiNER NER microservice is built on Python with FastAPI [25], served by Uvicorn. This is not an accident of tooling preference. GLiNER is a HuggingFace `transformers`-based model; its inference stack, tokenisers, and model-loading utilities are Python-native, and no equivalent stable interface exists on .NET. Everything else in the system, by contrast, benefits from the strong typing and reflection facilities of C#: the MCP server generates its tool schemas from C# attributes, and the Anthropic and Model Context Protocol SDKs used to call Claude and expose tools are both first-class .NET libraries. Splitting a system along the boundary where a specific runtime is genuinely the best fit for a specific responsibility, rather than forcing one language across every service, is standard practice in microservice design [22]; here that boundary falls exactly on the one component that does machine learning inference.

Redis [26] is the one piece of infrastructure shared, indirectly, across the ecosystem split. It holds no application code and is reachable only from the Anonymisation Service, which uses it to store the token-to-entity mapping generated for each anonymisation context. Because Redis is a plain key-value store reached over the network rather than a shared library or a shared database schema, the two ecosystems remain fully decoupled at the code level: the GLiNER service has no knowledge that Redis exists, and the C# services never touch it directly except through the `IAnonymizationContextStore` contract described in Section 4.2.3.

Dependency management follows each ecosystem's own convention rather than a project-wide standard. The three .NET projects declare their packages in `.csproj` files, and `dotnet`

`restore` runs automatically on build, so no separate install step is needed. The GLiNER Service declares its dependencies in `pyproject.toml` and uses `uv` [4] for resolution and environment management; `uv sync`, invoked automatically by `uv run`, resolves and installs everything on first run. Both conventions share the same underlying goal: a developer should be able to clone a service and run it without a manual dependency-installation step. The one point where the two ecosystems talk to each other in generated code, rather than over HTTP, is NSwag [29], which reads the Anonymisation Service's OpenAPI specification and generates a typed C# client consumed by both the MCP server and the MCP client.

4.2 The Reversible Anonymisation Pipeline

The anonymisation pipeline went through two implementation phases. The first, built on Microsoft Presidio, was designed before any concrete use case existed and did not survive contact with real requests. The second, built around GLiNER, replaced it once actual Italian business prompts exposed where the first version broke. Both phases share the same downstream requirement: whatever detects the entities, the resulting token-to-value mapping has to be stored somewhere the server can reach and the client cannot, which is what Section 4.2.3 describes.

4.2.1 Early Approach: Microsoft Presidio and its Limitations

Presidio [21] was the first detection engine integrated into the anonymisation service, chosen during the architecture design phase for the reasons given in Section 2.3.3: it is the leading open-source PII detection framework, its recognisers are predictable, and it runs entirely on local infrastructure with no external inference dependency. This choice was made before the company had supplied the three concrete use cases the system would eventually implement, so it was necessarily a decision made on general merit rather than on measured performance against real inputs.

Once the use cases arrived, the gap became apparent quickly. Presidio's recognisers are pattern- and rule-based, tuned against English-language, general-purpose PII, and neither the domain vocabulary (*cantiere*, *PULITO_DA*, *Capocantiere*) nor the register of the incoming prompts, informal spoken Italian with names embedded in operational phrasing rather than isolated in structured fields, matched what those recognisers were built to catch. Entities that a human would identify immediately, a person's name mid-sentence in a holiday request, were silently missed rather than flagged with low confidence. Presidio's recognisers do carry a per-pattern confidence score, but that score is a fixed value the recogniser's author assigns to a specific regular expression or context rule, not something inferred from the input; if no pattern exists for a given entity type or phrasing, there is no partial match and no score to threshold, only silence. Extending Presidio's coverage would have meant hand-writing new recognisers for each entity type and each way it could appear in an Italian operational sentence, which does not scale to the variability of real user input. This is the same structural limitation described abstractly in Section 2.3.3; here it surfaced as a concrete recall failure on the project's own test prompts, not as a theoretical concern.

4.2.2 The Pivot to GLiNER for Multilingual Italian NER

The anonymisation module was redesigned around GLiNER [34], a zero-shot NER model that accepts an arbitrary label set at inference time instead of a fixed, pre-trained schema. The deployed checkpoint, `urchade/gliner_multi-v2.1` [33], is trained for multilingual text

and requires no fine-tuning to work on Italian, which directly answers the failure mode described above: entity types are configured, not hand-coded per recogniser.

The GLiNER Service exposes a single `/analyze` endpoint that accepts a text string, a label list, a confidence threshold, and a language hint, and returns entity spans with their positions and scores. The default label set covers eight categories: person, organization, location, address, email, phone number, date, and time. In the current configuration, the Anonymisation Service calls this endpoint with a threshold of 0.45, set during initial configuration and left unchanged rather than arrived at through a systematic sweep. The value leans permissive by design: a missed entity is a privacy failure, while a spurious one only costs a minor, correctable annoyance in the reconstructed text, so erring toward recall over precision is the safer default in this specific asymmetry.

Raw GLiNER predictions are not returned to the caller directly. They pass through a three-stage filtering pipeline first. The normalisation stage discards any prediction whose offsets fall outside the source text or whose label is empty, and reconstructs the entity's text from the source string when it is missing from the raw prediction. The quality-filtering stage discards any entity below the confidence threshold, shorter than a minimum span length of three characters, or matching an Italian stopword drawn from a blocklist built specifically for this project's operational vocabulary (words such as *lista*, *risolvi*, *internamente*, terms that recur in the imperative, instruction-like phrasing of the system's own prompts and that GLiNER occasionally mistakes for entities). The deduplication stage resolves overlapping spans, such as *Giulia* and *Giulia Conti* both being flagged for the same text, by sorting candidates by score and then by span length, greedily keeping the best non-overlapping set, and restoring position order at the end. Each stage narrows recall in favour of precision only after the model has already been configured to favour recall, so the net effect is a low false-negative rate on real entities without the flood of false positives an untuned threshold would produce.

4.2.3 Deterministic Token Mapping and Redis Context Storage

Once GLiNER (or, previously, Presidio) has located an entity, the Anonymisation Service replaces it with a token of the form `<LABEL_NNNN>`: the entity's label in upper case, followed by a zero-padded four-digit counter scoped to that label within the current context. A prompt containing two distinct people therefore produces `<PERSON_0001>` and `<PERSON_0002>`, not two copies of the same token, which preserves the distinction between the two individuals through every step of the pipeline that follows, including the LLM's own reasoning about the request.

Substitution happens right to left: entities are sorted by their start offset, then replaced in the source text starting from the last one. This ordering is necessary because entity spans and their replacement tokens are rarely the same length; replacing an earlier entity first would shift the character offsets of every entity after it, corrupting the positions already computed by the detection stage. Processing from the end of the string backward means every substitution happens before the offsets of anything earlier in the text are touched.

The resulting token-to-value mapping is written to Redis [26] under a newly generated context identifier, through a narrow contract, `IAnonymizationContextStore`, with four operations: `Store` writes a new mapping, `Get` reads one without removing it, `Append` merges new tokens into an existing context for multi-turn sessions, and `Pop` reads and deletes the mapping in the same operation. Server-side deanonymisation always uses `Pop`, not `Get`, so a context is removed the instant it is actually consumed, rather than lingering after it has served its purpose. That guarantee covers only the path where `Pop` is actually called: no

time-to-live is configured on the Redis context itself, so a request that errors out before deanonymisation, or a client session that is never completed, leaves its mapping in Redis with no expiry. This is a deliberate scope decision rather than an oversight, discussed in full in Chapter 6: the system runs as a local proof of concept, with no persistence volume mounted on Redis, so every mapping is cleared the moment the process stops regardless, and what the internship needed to demonstrate was that pseudonymisation and its MCP-based orchestration work correctly end to end, not a production-grade retention policy.

4.3 Exposing Business Operations via MCP

Section 2.2.1 described MCP's three server-exposed primitives in the abstract; this section shows how the MCP server, `Antares_AI_McpServer`, turns ordinary C# classes into the *tools* primitive Claude calls. Section 4.3.1 covers schema generation, Section 4.3.2 covers the single entry-point tool every use case goes through, and Section 4.3.3 covers how that entry point's system prompt supports the privacy objective at the one place the model's behaviour cannot be checked by a type system.

4.3.1 C# Attributes and Reflection for Tool Schema Generation

Every class that exposes an MCP tool is decorated with `[McpServerToolType]`, and every method meant to be callable by Claude carries `[McpServerTool(Name = "...")]`. The `ModelContextProtocol.AspNetCore` package reads these attributes through reflection at startup and builds the JSON Schema tool definitions [2] that MCP requires, one schema per annotated method, derived from the method's own parameter types and names. A developer adding a new business operation writes an ordinary C# method and two attributes; the tool schema, its exposure over HTTP/SSE, and its registration with Claude's function-calling interface all follow from that annotation without a separate schema definition to keep in sync by hand.

Only two classes, `AntaresTools` and `JobTools`, are registered as MCP tool types via `.WithTools<T>()` in `program.cs`. The remaining tool classes are registered only as ordinary scoped services, injectable into `JobTools` but invisible to Claude:

- `DepartmentsTools`
- `IdentityTools`
- `AssetTools`
- `PersonTools`
- `AbsenceTools`
- `LookupTools`

This is a deliberate narrowing of the model's reachable surface, consistent with the first architectural property named in Section 2.1.2: bounding the model's agency by exposing only business-level operations while keeping data access inside deterministic code that the model never calls directly.

4.3.2 The `ask_antares` Routing Tool

`AntaresTools.AskAntares` is the only tool the MCP client ever calls. It takes two parameters, `user_input` (the anonymised prompt, which may contain tokens such as `<PERSON_0001>`) and an optional `context_id` (the anonymisation session identifier, forwarded so that sub-tools can trigger server-side deanonymisation later in the call chain). Internally it builds an Italian-language system prompt that casts Claude in a single role: a semantic router that must pick exactly one of the five sub-tools available to it, forward `context_id` unchanged, never restate or reveal PII, and always return the tool's own hardcoded confirmation string rather than a freeform summary of the data it touched. It then calls `IChatClient.GetResponseAsync()`, the .NET binding for Claude's schema-driven tool-use contract [3], with function invocation enabled and the sub-tool list attached, and returns Claude's response text unmodified. The call runs at temperature 1.0, a configuration value set during initial setup and left unchanged; because sampling at that temperature is not deterministic, repeated invocations of the same anonymised prompt are not guaranteed to select the same sub-tool or extract identical parameters, which Chapter 6 identifies as a methodological limitation of the validation carried out in this thesis.

The five sub-tools Claude can select from are:

- `GetDepartments`
- `GetDepartement`
- `GetNearestWorkersToCantiere`
- `GetColleaguesOfPerson`
- `ReportHolidayRequest`

The first two are leftovers from an early prototyping phase; the latter three are the three use cases this thesis covers in Chapter 5. Claude's role stops at selecting one of these five names and extracting the parameters to call it with. Everything downstream, deanonymisation, GUID resolution, ranking, report generation, happens inside `JobTools`, in code the model never executes and cannot see the output of beyond the final confirmation string.

Calls to the Claude API can fail transiently under load. `AskAntares` retries on an `overloaded_error` response up to four times with exponential backoff (2, 4, then 8 seconds between attempts) [5], and only after all four attempts fail does it return a hardcoded Italian error string to the caller. This bounds how long a single request can block on a transient provider-side failure without silently dropping it after the first error.

4.3.3 Strict Prompting for Privacy Enforcement

The system prompt inside `AskAntares` is the one point in the architecture where privacy enforcement is a matter of instruction rather than structural design, and it is worth being explicit about why that gap exists. Every component upstream of Claude, the anonymisation pipeline, the token substitution, the Redis-backed context store, is designed to keep PII from reaching the model, per the privacy-by-design argument of Section 3.1, though that protection still depends on the detection layer having correctly identified the entity in the first place. Claude itself, however, is not code the project controls; it can only be instructed, not verified line by line the way `JobTools` can. The system prompt therefore embeds three explicit constraints: keep any anonymised token exactly as received rather than paraphrasing or translating it, never introduce a name, phone number, or address that was not already

present in the input, and always forward `context_id` to whichever sub-tool is selected so that server-side deanonymisation can proceed.

These constraints are a mitigation, not a proof. A model that ignored its system prompt entirely could in principle violate all three, and the tool-result feedback loop this system relies on is the same one identified as the primary attack surface for indirect prompt injection [14] in Section 2.2.2. `AskAntares` returns Claude's response text as-is; nothing strips or checks it afterward, so a model that decided to fabricate or restate PII in its own words would not be caught at that boundary. What actually limits the damage is upstream of the prompt entirely: as long as the anonymisation layer correctly detected the entities in a given request, Claude has no real PII in its context to leak. `JobTools` deanonymises parameters and queries real data only after Claude has already selected a tool and returned its arguments; the sub-tool's return value back to Claude is always the same hardcoded success string, never the report it just generated, and that report, containing real names and phone numbers, is written to a server-local log file Claude has no access to. For any request where detection succeeded, a misbehaving model would have no sensitive data available to misuse, regardless of what the system prompt tells it to do with data it was never given.

4.4 Use Case Orchestration in `JobTools`

The three sub-tools Claude can route to, described in Section 4.3.2, all live in `JobTools` and all follow the same five-step shape. First, deanonymise whichever natural-language parameter Claude extracted, a person's name or a cantiere's name, via `/deanonymize-known`; if `context_id` is null or the call fails, the parameter passes through unchanged rather than blocking the request, so the tool degrades gracefully when anonymisation was not used upstream. Second, resolve the relation-type and category GUIDs the operation needs, since the Miorelli API addresses everything by GUID rather than by name. Third, query the lower-level Miorelli tools to assemble the relevant dataset, scoped to whichever cantieri the caller is authorised to see. Fourth, apply the use case's actual business logic: ranking, an authorisation check, or absence cross-referencing. The order of steps three and four is not fixed across all three use cases: Use Case C, described in Section 4.4.3, runs its authorisation check before assembling the main dataset rather than after, since there is no reason to query absence data for a request the caller was never entitled to make. Fifth, write a full report containing the real, deanonymised data to a server-local log file and return the same hardcoded Italian confirmation string regardless of what the report contains.

Steps one and five are where the architectural design objective from Chapter 3 is pursued in practice, not just steps two through four where the actual business logic happens. Step one is the only place real data re-enters the system after anonymisation; step five is the only place it leaves again, and it leaves toward disk, never back toward Claude or the client.

4.4.1 Use Case A: Nearest Workers to a Cantiere

`GetNearestWorkersToCantiere` takes a cantiere name and an optional search radius (15 km by default). After deanonymising the name, it resolves the caller's identity, the `PULITO_DA` relation type, and the `SedePrincipale` item category, then calls `GetAuthorizedCantieri()` and matches the deanonymised name against the caller's authorised sites. Once the target cantiere is found, `GetPeopleByRelationOnAsset` returns every worker linked to it through `PULITO_DA`, and each worker's details, including their residential coordinates, are fetched individually.

Ranking those workers by distance needs no additional service call, only the Haversine formula [28], which gives the great-circle distance between two points on a sphere from their latitude and longitude. `CalculateDistanceAndRank` implements this step directly. At the scale of workers travelling to a single cantiere, a few to a few dozen kilometres, the curvature of the Earth is not negligible in the way a flat-plane distance approximation would assume, so the spherical formula is the correct choice even though the search radius is small. Workers with no coordinates on file are sorted last rather than excluded, and `FilterByRadiusKm` then drops anyone outside the requested radius. The result is split into *Attivi* and *NonAttivi* before the full report, names, phone numbers, and computed distances, is logged. The report generation also covers three edge cases, each producing a distinct log entry: the deanonymised name matches no known cantiere, neither the cantiere nor the associated service is recognised, or the cantiere is recognised but falls outside the caller's own authorised perimeter. The response Claude and the client receive is the same regardless of which branch fired.

4.4.2 Use Case B: Colleagues of a Person

`GetColleaguesOfPerson` answers two different questions about the same deanonymised person in a single call: who their colleagues are, and who else works nearby without being one. After resolving the person by name, `GetColleaguesOfPerson` on the Miorelli side returns everyone who shares at least one cantiere with them under the `PULITO_DA` relation, which is a relational query, not a spatial one. The method then scans every cantiere the caller is authorised to see and collects workers who do *not* share any site with the target person, that is, everyone the previous step did not already return. `CalculateDistanceAndRank`, the same routine used in Section 4.4.1, ranks that non-colleague pool by distance from the person's residence, and only the closest five are kept. The report logs both lists, direct colleagues and the five nearest non-colleagues, before returning the hardcoded confirmation string.

4.4.3 Use Case C: Holiday Request Analysis

`ReportHolidayRequest` is the most involved of the three, because it has to decide who is even allowed to ask before it computes anything. After deanonymising the target person's name and resolving the `RESPONSABILE_DI_RIFERIMENTO` and `AUTORIZZATORE` relation types, the method checks that the caller satisfies at least one of three conditions: managing the person within their own `PULITO_DA` perimeter, being their Responsabile, or being their Autorizzatore. This authorisation gate runs entirely in deterministic C# code, before any absence data is touched, rather than being left to Claude's judgement; Claude has already exited the loop by the time this check happens, since its role stopped at selecting the tool and extracting the person's name (Section 4.3.2).

Once authorised, the method builds the full worker pool across every cantiere the caller can see and identifies which of those cantieri the target person actually works at as a `PULITO_DA` worker. For each of those cantieri, it calls `GetAbsencesForPerson` against every worker on site to find who else is absent that day, separates absent colleagues from the rest, and finds non-colleagues within 15 km, flagging which of those are also absent. The report this produces is per-cantiere rather than a single flat list, since a substitute needs to be both physically close and actually available, and availability only makes sense in the context of one specific site's absence record on one specific day. As with the other two use cases, the caller and Claude see only the confirmation string; the per-cantiere breakdown exists solely

in the server-side log.

4.5 Backend Integration and Data Abstraction

Every use case described in Section 4.4 needs real workforce data: persons, cantieri, relations, absences. None of the tool classes that fetch this data talk to the Antares/Miorelli backend directly. They talk to `IMiorelliService`, and for the entire duration of the internship, the concrete implementation behind that interface was a mock, not the production system.

4.5.1 The `IMiorelliService` Interface

`IMiorelliService` is the single interface every data-access tool class depends on for person, cantiere, relation, and absence data:

- `PersonTools`
- `AssetTools`
- `AbsenceTools`
- `LookupTools`
- `IdentityTools`

None of these classes constructs an HTTP request or knows the shape of the Antares REST API; they call a method on the interface and get back a strongly typed result. Depending on an abstraction rather than a concrete backend is the dependency inversion principle [19] applied at the one seam in the system that was guaranteed to change: high-level business logic in `JobTools` and the tool classes does not depend on low-level details of how Miorelli data is actually fetched, and the reverse dependency, a concrete implementation satisfying the interface's contract, can be swapped without touching a single line of the code that calls it.

Not every tool-class method is a passthrough to the interface. `CalculateDistanceAndRank`, defined on `PersonTools` and used throughout Section 4.4, is pure computation over data `IMiorelliService` already returned. The interface itself only ever hands back persons, cantieri, and relations, not a ranking of them.

4.5.2 The Stub/Mock Pattern

`MiorelliServiceStub` is the only implementation of `IMiorelliService` that existed during the internship. It returns data from a small, hand-built in-memory dataset, the same fictional workers and cantieri (Giulia Conti, the Bergamo site) that appear in the use case prompts throughout Chapter 5, rather than querying any real database.

This was a deliberate scope decision, not an oversight. The production Miorelli database holds a large volume of personnel data, and building the anonymisation layer and its integration with a real, sensitive data source at the same time would have spread the team's attention across two hard problems at once, with actual PII at risk if either was rushed. The team chose to get the anonymisation and orchestration architecture right against a safe mock dataset first and treat real backend integration as a separate, later step, prioritising correctness and data safety over finishing the full integration within the internship's timeframe.

The stub also models failure, not just data. Where a piece of mock data is genuinely absent, it throws `MiorelliClientNotAvailableException` rather than returning an empty or null result. `IdentityTools.ResolveCallerIdentity` catches this exception and falls back to a hardcoded default identity (Paolo Ricci, Service Manager) instead of letting the failure propagate to the caller. The mechanism is a plain try/catch around a single call site, not a dedicated type woven into the domain model, but the motivation is the one behind Fowler's Special Case pattern [13]: a caller should not have to handle an exceptional outcome specially when a well-formed, ordinary-looking value satisfying the same contract can stand in for it instead.

4.5.3 NSwag Client Generation and the Migration Path

The path to a real backend is not a from-scratch integration; most of it already exists and sits unused behind the stub. `program.cs` registers a named `AntaresGeneratedClient` HTTP client pointed at the real Antares API (`ApiSettings:GeneratedClientBaseUrl`), and an `AntaresAPIClient`, generated by NSwag from that API's OpenAPI specification, the same code-generation approach already used for the Anonymisation Service client in Section 4.1. Basic-auth credentials for the real API are already read from `ApiSettings:ApiKey` and `ApiSettings:ApiSecret` via `dotnet user-secrets`, configured and ready even while the stub is what actually runs.

Migrating to the real backend means writing one new class that implements `IMiorelliService` using `AntaresAPIClient` instead of the in-memory dataset. After that, `program.cs`'s dependency injection registration changes on a single line: `IMiorelliService` currently resolves to `MiorelliServiceStub`, and it would resolve to the new class instead. Every tool class, every use case in `JobTools`, and every MCP tool schema built from those classes stays exactly as written, because none of them ever depended on which implementation of `IMiorelliService` they were talking to. This is the payoff of Section 4.5.1's abstraction: the cost of deferring backend integration was paid once, in designing the interface correctly, rather than being owed again at the point the real system finally arrives. That one-line swap is necessary for the migration, not sufficient for it: writing `AntaresMiorelliClient` itself, propagating the real caller's identity instead of the hardcoded fallback in `IdentityTools` (Section 4.3.2), authentication, role-based access control, integration testing against the real tenant, and the operational hardening any of that would require, all remain open work the abstraction does not do for free. Antares AI, throughout this thesis, is a prototype validated against that mock backend, not the production system it is designed to eventually front.

Chapter 5

Use Cases and Validation

Chapters 3 and 4 described the architecture and the implementation of Antares AI's three business use cases. This chapter validates them, against a fixed mock dataset and a purpose-built evaluation harness that exercises the running system through its real interfaces rather than its internal code paths. Section 5.1 summarises the workforce management domain the three use cases operate on. Section 5.2, Section 5.3, and Section 5.4 report end-to-end correctness for Use Cases A, B, and C respectively, against an independent reference implementation built separately from the system under test. Section 5.5 closes the chapter with the two cross-cutting quantitative results the privacy and correctness claims of the previous chapters were left resting on: how often PII detection fails, whether that failure ever reaches the LLM payload, and how consistently the routing model selects the correct tool under its configured sampling temperature.

5.1 Workforce Management Domain Overview

Section 3.2 already introduced the Antares/Miorelli domain: *persone*, *cantieri*, and *ticket* as the three core entity types, and the `PULITO_DA`, `RESPONSABILE_DI_RIFERIMENTO`, and `AUTORIZZATORE` relations that structure the three use cases this chapter validates. This section does not repeat that description; it fixes the concrete dataset and scope every result below was measured against.

Every use case validated in this chapter runs against `MiorelliMockDataProvider.cs`: ten persons, three cantieri (Bergamo, Brescia, Mantova), the `PULITO_DA` work assignments linking persons to cantieri, and five absence tickets, all dated within 2026-03-28 to 2026-04-10. No production Miorelli backend and no real employee data were touched at any point in this validation; every number in this chapter was produced against this fixed, fictional dataset, consistent with the prototype scope stated in Section 4.5.

5.2 Use Case A: Spatial Querying for Construction Sites

Use Case A is validated against four test cases exercising `get_nearest_workers_to_cantiere`: the canonical Bergamo prompt already traced in Section 3.6.1, one case each for the other two cantieri, and one case naming a cantiere that does not exist, to exercise the not-found path.

5.2.1 Haversine Distance and Radius Filtering

Correctness here means agreement between the live system's output and an independently computed expected result, not agreement between the system and itself. The reference used for this comparison is a hand-built mirror of the mock dataset with its own Haversine implementation, written separately from `PersonTools.CalculateDistanceAndRank` (Section 4.4.1) rather than sharing code with it, so a match is a real check rather than the system confirming its own arithmetic. Before running the full test set, the formula itself was spot-checked against a live server report: for the canonical Bergamo prompt at a 15 km radius, the server logged two workers at 0.9 km and 1.9 km respectively, and the independent calculation matched both distances to within 0.1 km.

Across four cases, run twice each, every case matched the independent reference on every repetition. The Brescia case correctly returns the two `PULITO_DA` workers assigned there; the not-found case correctly reports no matching cantiere. The Mantova case is a genuine negative result rather than a placeholder: both workers assigned to that cantiere live outside the 15 km search radius, and the independent reference agrees that the correct output is an empty result, not a system failure.

5.2.2 Execution Flow and Prompt Handling

`ask_antares` returns only free text to its caller; it does not itself expose which sub-tool ran. What does expose it, without any change to server code, is the report file each `JobTools` invocation already writes to `logs/` (Section 4.4): a use-case-tagged, timestamped file whose name identifies which of the three tools actually executed. The validation harness snapshots that directory before and after each `ask_antares` call and identifies the new file by name, giving a reliable, zero-instrumentation way to observe routing decisions from outside the system. Section 5.5.2 reports the routing accuracy and consistency results this technique produced, for all three use cases together, since the same methodology and the same headline numbers apply across UC-A, UC-B, and UC-C.

5.3 Use Case B: Colleague Identification and Proximity Ranking

Use Case B is validated against four test cases exercising `get_colleagues_of_person`: the canonical Giulia Conti prompt, two further named persons chosen to exercise different colleague-set sizes, and one nonexistent person to exercise the not-found path. The same independent-reference methodology as Use Case A applies here, comparing colleague name sets rather than distances: the reference computes everyone sharing at least one `PULITO_DA` cantiere with the target person, independently of the live system.

Building the evaluation harness surfaced a methodological pitfall worth recording rather than silently fixing. `JobTools`' UC-B report contains two structurally similar row blocks: the person's actual colleagues, and the five nearest non-colleagues, a deliberately disjoint set used for substitute-finding rather than colleague identification. An early version of the log parser used a single pattern to extract name rows and matched both blocks indiscriminately, which made every case look like a mismatch, since the observed set always included several unrelated extra names. The fault was in the evaluation harness, not in the system under test; the fix was scoping extraction to the text between the two section headers before re-running. It is included here because it generalises: log-scraping evaluation of a black-box

system needs the same care about a report's internal structure as any other parser would, and a first result that looks wrong is not automatically evidence that the system is wrong.

Across four cases, run twice each, every case matched the independent reference on every repetition, including one case where the observed and expected colleague sets differed only in row order, not in membership.

5.4 Use Case C: Holiday Request and Absence Analysis

5.4.1 Workflow Logic: Colleagues, Substitutes, and Authorization

`JobTools.ReportHolidayRequest` determines which colleagues are absent by comparing each ticket's date range against `DateTimeOffset.UtcNow.Date`, the wall-clock date at the moment the tool runs (confirmed by reading the source rather than assumed). Every mock absence ticket in the dataset described in Section 5.1 falls between 2026-03-28 and 2026-04-10. This validation run executed on 2026-08-06, outside that window, so every UC-C test case correctly reports zero absent colleagues on the day it ran, on both the live system and the independent reference, which uses the identical wall-clock comparison. This is a deterministic consequence of running a fixed, dated mock dataset outside its own date range, not a detection or logic failure, and it means this particular run did not exercise the branch of UC-C's logic that identifies an absent colleague and searches for a substitute. That is stated here as a limitation of *when* this run was executed, not of the system: the dataset and harness are unchanged by the date, and a re-run during 2026-04-01 to 2026-04-10 would exercise the substitute-finding branch directly.

The three non-edge-case test cases were chosen specifically to exercise the three distinct ways `ReportHolidayRequest` authorises a caller (Section 4.4.3): as the target person's `RESPONSABILE_DI_RIFERIMENTO`, as their `AUTORIZZATORE`, and as a manager reaching them through the caller's own `PULITO_DA` perimeter. All three authorised successfully, confirmed directly in each case's server-side log.

Across four cases, run twice each (three authorization paths plus one nonexistent-person not-found case), every case matched the independent reference on every repetition: twelve of twelve end-to-end cases across all three use cases, combined, matched in full. The same log-parsing pitfall described in Section 5.3 recurs here in reverse: the report's list of absent colleagues sits directly above a list of nearby non-colleagues flagged present or absent, and the same section-scoping fix was required before the parser produced correct results.

5.5 Privacy and Functional Validation

The preceding three sections validated business-logic correctness. This section validates the two claims Chapters 3 and 4 left open pending measurement: whether PII actually reaches the LLM payload in practice, and how accurately and consistently the routing model selects the correct tool given its configured sampling temperature. Both were measured against the running system through its real interfaces, not against its internal code paths, over a fifty-prompt Italian test set built for this purpose.

5.5.1 Auditing the LLM Context Boundary

Section 3.1 states the structural claim directly: PII isolation holds only for entities the detection layer actually finds, and the risk from a missed entity was left for this chapter to

address, either by reporting measured detection performance or by stating explicitly that no such measurement was carried out. It was carried out.

`AntaresTools.AskAntares` sends Claude exactly two messages per call: a static Italian system prompt, unchanged except for the `context_id` it carries, and a user message equal to whatever `user_input` string the caller supplied. Because the validation harness is that caller, it constructs `user_input` itself, which makes the exact literal payload Claude receives fully reconstructable client-side, with no server-side logging, instrumentation, or code change of any kind. That reconstructed payload was scanned, for all fifty test prompts, for any of thirty-nine known real PII values drawn from the mock dataset: full names, phone numbers, email addresses, and full site or company descriptors.

Two leaks were found, out of fifty prompts scanned: an email address in one prompt and a different email address in another, both cases where GLiNER failed to tag the address in its specific sentence context. Both leaks correlate with a GLiNER false negative reported in Section 5.5.2; zero leaks were found that do not correlate with a detection miss. PII reached the Claude payload if and only if GLiNER had already failed to detect it, in every observed case. This is the direct empirical measurement behind the structural argument of Section 3.1: the anonymisation layer has no independent bypass path of its own, and the privacy property this architecture provides reduces entirely to detection recall. It also means the false-negative list below is not only a statement about NER quality; each entry in it is a demonstrated, specific PII leak path, not a hypothetical one.

5.5.2 Routing Accuracy and NER Performance in Italian

PII detection. Fifty hand-built Italian prompts, carrying eighty-two labelled entity spans, were evaluated against the live `/anonymize` endpoint at its production configuration (Section 4.2.2): the real GLiNER-based detection path, threshold 0.45, the full eight-label set. Two metrics are reported: *coverage*, whether a PII substring was tokenised at all regardless of the label predicted for it, which is the property that actually determines whether PII can leak; and *strict*, standard span-and-label NER evaluation. Coverage precision, recall, and F1 were 0.938, 0.915, and 0.926 respectively; strict precision, recall, and F1 were 0.840, 0.829, and 0.834.

Seven entities were missed entirely under the coverage metric. Two were email addresses missed mid-sentence, both already identified in Section 5.5.1 as the two payload leaks. Four were Italian date-range expressions of the form *"dal X al Y [aprile]"*, and one was a bare cantiere-city reference embedded inside a compound site descriptor. The date-range pattern accounts for four of the seven false negatives, the single largest recurring failure mode in this test set, and is reported here as a concrete, actionable finding rather than folded into an aggregate score: at its current threshold and label configuration, GLiNER's weakness on this test set is specific to Italian date-range phrasing, not PII detection generally. Five false positives were also observed, none of them random noise: two are generic temporal expressions plausibly mislabelled under a recall-favouring threshold, two are bare city or region names used generically inside a prompt built specifically to probe that behaviour, and one is a genuine spurious organisation detection. The gap between strict recall (0.829) and coverage recall (0.915) is consistent with the design rationale already stated in Section 4.2.2: the 0.45 threshold catches more entities than it assigns correct labels to, which is the expected shape of a configuration deliberately tuned toward recall over precision.

Routing accuracy and temperature consistency. Twenty-one prompts, six per use case plus three out-of-scope negative controls, were each run five times through the full anonymise-then-route pipeline, since the routing model runs at temperature 1.0 (Sec-

tion 4.3.2) and a single pass cannot establish whether that setting produces run-to-run variation. Measured accuracy was 95.2%, 100 of 105 trials correct, and every one of the twenty-one prompts selected the same outcome on all five repetitions: on this test set, at this prompt style and this system-prompt design, temperature 1.0 produced no observed run-to-run routing variance, a genuinely open question this component was built specifically to answer.

The one measured discrepancy is not a routing failure, and reporting only 95.2% would misstate what was actually found. One prompt, requesting a holiday ticket for a person with no `PULITO_DA` cantiere assignment in the mock dataset, registered as zero correct out of five trials under the log-diffing detection method described in Section 5.2.2. Inspecting the response text directly shows Claude selected `report_holiday_request` correctly on every one of the five repetitions; the tool itself returned a legitimate business-logic failure message, since that person genuinely has no eligible cantiere, and that failure path exits `JobTools.ReportHolidayRequest` before the code that writes the use-case log file the detection method relies on. Log-diffing cannot distinguish a tool that was never invoked from one that was invoked and exited early, before logging: a blind spot in the observability method, not a defect in the routing decision it was measuring. True routing accuracy on this dataset is very likely 105 of 105, effectively 100%; 95.2% is what this specific black-box detection method measures, with its one discrepancy fully accounted for. Both figures are reported here together, rather than the raw 95.2% alone, because the explanation is itself part of what the measurement found, not an excuse for it.

A further, unrelated finding surfaced in the same routing data, worth connecting back to the tokenisation example corrected in Section 3.6.1. For the canonical UC-A prompt, Claude's response shows it invoked `get_nearest_workers_to_cantiere` three times within a single `ask_antares` call, once per GLiNER's three fragmented location tokens, rather than treating them as one compound site descriptor. This did not register as a routing error under the detection method used here, since the correct tool was still selected and only the most recent log file per repetition was captured, and it is harmless in this specific case, since all three lookups resolve to the same cantiere and the end-to-end results in Section 5.2 already confirm the correct final output. It is reported here as a limitation rather than corrected within this validation pass: fragmented anonymisation does not only risk mislabelling an entity, it can also change how many times, and with what parameters, the model invokes a business tool, and a differently fragmented prompt could plausibly produce more than one genuinely incorrect lookup instead of three redundant correct ones.

Chapter 6

Discussion and Future Work

This chapter reflects on what the implementation and validation described in the previous chapters actually established, and where the current system falls short of what running in production would require. Section 6.1 weighs the security properties the anonymisation pipeline provides against the architectural complexity it adds, including the prototype-scope gaps already flagged in earlier chapters rather than solved problems. Section 6.2 separates what typed schemas and deterministic code actually guarantee from what was measured, and names the routing model's temperature setting as an open methodological question rather than an implicit assumption. Section **??** discusses what remains before the mock Miorelli backend can be replaced with the production system. Section **??** considers how the Redis-based context store would need to change under concurrent, multi-tenant load. Section **??** closes the chapter with two possible extensions: multimodal (voice) input and locally hosted models.

6.1 Security vs. Complexity Trade-offs

The anonymisation pipeline's privacy protections stop short of what a production deployment would require in two related places. The Redis context holding token-to-entity mappings (Section 4.2.3) has no time-to-live configured. A mapping is removed only when `Pop` is actually called during a successful server-side deanonymisation; a request that errors out before that point, or a client session that is abandoned mid-flow, leaves its mapping in Redis with no expiry, bounded in practice only by the process's own lifetime. The per-request reports written to `JobTools`'s local `logs/` directory (Section 4.4) sit on the other side of the same gap: they contain real names, phone numbers, and distances, are never transmitted anywhere and were never intended to be, but they are also an unencrypted, access-uncontrolled, retention-unmanaged record on local disk.

Neither gap was addressed during the internship, and that was a deliberate scope decision rather than an oversight. The system runs as a local proof of concept: Redis has no persistence volume mounted, so every context is cleared the moment the container stops regardless of whether `Pop` ever ran, and the log files exist on a machine no one outside the development team has access to. What the internship needed to demonstrate was that reversible pseudonymisation and its MCP-based orchestration work correctly end to end, not that the system already meets the retention and access-control standards a production deployment would be held to. Closing this gap would mean configuring an explicit time-to-live on the Redis context matched to the maximum reasonable lifetime of a single request,

defining a retention and deletion policy for the log files, and encrypting them at rest; none of this requires redesigning the architecture described in Chapter 3, only additional operational hardening on top of it.

6.2 Validation Methodology Limitations

Two claims elsewhere in this thesis rest on single-pass evidence rather than systematic measurement, and it is worth stating that distinction plainly rather than leaving it implied. Typed tool schemas (Section 4.3.1) guarantee that a call Claude makes is well-formed, a valid tool name with parameters of the right type. They guarantee nothing about whether that call was the right one. Whether Claude selected the correct sub-tool for a given request, or extracted the correct parameter from an anonymised prompt, is a separate question no schema can answer, and no systematic test suite in this thesis answers it either: the worked examples in Chapter 5 show each use case executing correctly once, not routing accuracy measured across a representative set of Italian prompts.

The same gap compounds with a second one. `AskAntares` calls Claude at temperature 1.0 (Section 4.3.2), a sampling setting under which the same prompt is not guaranteed to produce the same output on repeated calls. A single successful trace, of the kind Chapter 5 presents, says nothing about whether that same prompt would route identically on a second or third attempt. Establishing routing accuracy as a general property of the system, rather than a property observed once per use case, would require running each test prompt multiple times and reporting both the correctness of each run and the consistency across runs, which this thesis did not do.

Chapter 7

Conclusions

7.1 Summary of the Work

7.2 Final Remarks on Enterprise AI Adoption

Bibliography

[1] Federico Albanese, Pablo Ronco, and Nicolás D'Ippolito. *Anonymous-by-Construction: An LLM-Driven Framework for Privacy-Preserving Text*. `https://arxiv.org/abs/2603.17217`. arXiv preprint arXiv:2603.17217. Accessed: July 2026. 2026.

[2] Anthropic. *Model Context Protocol Specification*. `https://modelcontextprotocol.io/specification`. Open protocol for connecting LLM clients to external tools and data sources. Accessed: June 2026. 2024.

[3] Anthropic. *Tool Use with Claude*. `https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview`. Claude API documentation for schema-driven function calling, invoked via the Anthropic.SDK in `AntaresTools`. Accessed: July 2026. 2024.

[4] Astral Software Inc. *uv: An Extremely Fast Python Package and Project Manager*. `https://docs.astral.sh/uv/`. Used for dependency resolution and environment management in the GLiNER Service. Accessed: July 2026. 2024.

[5] Marc Brooker. *Exponential Backoff and Jitter*. `https://aws.amazon.com/blogs/architecture/exponential-backoff-and-jitter/`. AWS Architecture Blog. Accessed: July 2026. 2015.

[6] Tom Brown et al. "Language models are few-shot learners". In: *Advances in Neural Information Processing Systems* 33 (2020), pp. 1877–1901.

[7] Hendrik Bünder. "Decoupling language and editor — the impact of the language server protocol on textual domain-specific languages". In: *Proceedings of the 7th International Conference on Model-Driven Engineering and Software Development (MODELSWARD)*. 2019, pp. 129–140. DOI: `10.5220/0007556301290140`.

[8] Ann Cavoukian. *Privacy by Design: The 7 Foundational Principles*. Tech. rep. Information and Privacy Commissioner of Ontario, Canada, 2009.

[9] Vincenzo Dalia. "Workflow assistito da AI per la creazione documentale: un approccio conversazionale basato su RAG". Relatori: Luigi De Russis, Daniele Sabetta; sessione di laurea Aprile 2025. Accessed: July 2026. Tesi di Laurea Magistrale in Ingegneria Informatica. Politecnico di Torino, 2025.

[10] Jacob Devlin et al. "BERT: Pre-training of deep bidirectional transformers for language understanding". In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Vol. 1. 2019, pp. 4171–4186.

[11] Umberto Eco. *Come si fa una tesi di laurea*. La Nave di Teseo Editore spa, 2017.

[12] European Parliament and Council of the European Union. *Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation)*. Official Journal of the European Union, L 119, pp. 1–88. 2016.

[13] Martin Fowler. *Special Case*. `https://martinfowler.com/eaaCatalog/specialCase.html`. From *Patterns of Enterprise Application Architecture*. Accessed: July 2026. 2002.

[14] Kai Greshake et al. "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection". In: *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*. 2023, pp. 79–90.

[15] Shilong Hou et al. *A General Pseudonymization Framework for Cloud-Based LLMs: Replacing Privacy Information in Controlled Text Generation*. `https://arxiv.org/abs/2502.15233`. arXiv preprint arXiv:2502.15233. Accessed: July 2026. 2025.

[16] Xinyi Hou et al. *Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions*. `https://arxiv.org/abs/2503.23278`. arXiv preprint arXiv:2503.23278. Accessed: July 2026. 2025.

[17] Stefan Larson et al. "An Evaluation Dataset for Intent Classification and Out-of-Scope Prediction". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. 2019, pp. 1311–1316. DOI: `10.18653/v1/D19-1131`.

[18] Daniele Mansillo. "Multimodal RAG for Slide Presentations with Synthetic Data Generation and Anonymization". Relatori: Daniele Apiletti, Simone Monaco. Accessed: July 2026. Tesi di Laurea Magistrale in Data Science and Engineering. Politecnico di Torino, 2025.

[19] Robert C. Martin. "The Dependency Inversion Principle". In: *C++ Report* 8.6 (1996), pp. 61–66.

[20] Microsoft. *.NET 10*. `https://dotnet.microsoft.com`. Cross-platform application framework used for the MCP server, anonymisation API, and MCP client. Accessed: June 2026. 2025.

[21] Microsoft. *Presidio — Data Protection and De-identification SDK*. `https://github.com/microsoft/presidio`. Open-sourced by Microsoft in 2018. Accessed: June 2026. 2018.

[22] Sam Newman. *Building Microservices: Designing Fine-Grained Systems*. 2nd. O'Reilly Media, 2021.

[23] OpenAI. *Model Card for OpenAI Privacy Filter*. `https://cdn.openai.com/pdf/c66281ed-b638-456a-8ce1-97e9f5264a90/OpenAI-Privacy-Filter-Model-Card.pdf`. Released April 22, 2026 under the Apache 2.0 licence. Sparse Mixture-of-Experts token-classification model for PII detection. Benchmark F1 figures (Table 1, Section 7.4.1, p. 14): 96.0% token-level F1 on PII-Masking-300k (baseline), 97.4% on the OpenAI-corrected version. Accessed: July 2026. 2026.

[24] Shishir G Patil et al. "Gorilla: Large Language Model Connected with Massive APIs". In: *arXiv preprint arXiv:2305.15334* (2023).

[25] Sebastián Ramírez. *FastAPI*. `https://fastapi.tiangolo.com`. Modern, high-performance web framework for building APIs with Python. Accessed: June 2026. 2018.

[26] Salvatore Sanfilippo. *Redis*. `https://redis.io`. In-memory data structure store used for ephemeral session context storage. Accessed: June 2026. 2009.

[27] Timo Schick et al. "Toolformer: Language models can teach themselves to use tools". In: *Advances in Neural Information Processing Systems* 36 (2023), pp. 68539–68551.

[28] Roger W. Sinnott. "Virtues of the Haversine". In: *Sky and Telescope* 68.2 (1984), p. 159.

[29] Rico Suter and NSwag contributors. *NSwag: The Swagger/OpenAPI toolchain for .NET*. `https://github.com/RicoSuter/NSwag`. Repository created September 2015. Used for automatic C# client generation from OpenAPI specifications. Accessed: June 2026. 2015.

[30] Ashish Vaswani et al. "Attention is all you need". In: *Advances in Neural Information Processing Systems* 30 (2017).

[31] Jason Wei et al. "Emergent abilities of large language models". In: *Transactions on Machine Learning Research* (2022).

[32] Shunyu Yao et al. "ReAct: Synergizing reasoning and acting in language models". In: *The Eleventh International Conference on Learning Representations*. 2023.

[33] Urchade Zaratiana. *gliner_multi-v2.1 Model Card*. `https://huggingface.co/urchade/gliner_multi-v2.1`. Multilingual GLiNER checkpoint used by the anonymisation pipeline for Italian NER. Apache 2.0 licence. Accessed: July 2026. 2024.

[34] Urchade Zaratiana et al. "GLiNER: Generalist model for named entity recognition using bidirectional transformer". In: *arXiv preprint arXiv:2311.08526* (2023).

List of Figures

List of Tables